

DILoCC: An approach for Distributed Incremental Learning across the Computing Continuum

Giovanni Cicceri^{*†}, Giuseppe Tricomi^{*†}, Zakaria Benomar^{*},
Francesco Longo^{*†}, Antonio Puliafito^{*†}, Giovanni Merlino^{*†}

^{*} Engineering Department, University of Messina, Italy

{gcicceri,gtricomi,zbenomar,flongo,apuliafito,gmerlino}@unime.it

[†]CINI: National Interuniversity Consortium for Informatics, Rome, Italy

Abstract—The Internet of Medical Things (IoMT), combined with interconnected wearable devices and medical-grade sensors, can play an essential role in healthcare evolution. By exploiting the data generated by the plethora of interconnected devices (vital parameters, location-based info, patients activity and more), advanced ICT systems can be put in place with predicting capabilities. This way potentially critical situations, that may evolve in serious complications to patients' well-being, can be promptly recognized and successfully addressed, first of all, to save lives and secondarily to limit economical damages. To support continuous patient monitoring in public and private healthcare, this paper proposes “DILoCC”, an architecture to manage wearable devices, sensors and applications, that uses a Distributed Incremental Learning (DIL) approach to exploit cooperation among the sensing devices and increase the overall system efficiency through the mitigation of “Catastrophic Forgetting” consequences.

Index Terms—Incremental Learning, Fog/Edge computing, e-Health, Catastrophic Forgetting, IoMT, Computing Continuum.

I. INTRODUCTION

The world where we are living is becoming increasingly digital by virtue of the rise of the Internet of Things (IoT). Our whereabouts are turning progressively into smart environments, ranging from individual houses, to buildings, schools, hospitals, or even factories. This transformation process is involving gradually all the aspects encompassing our daily routine, including, e.g., medical treatments that people get at home or in hospitals; in the latter scenario, employing devices and systems going under the Internet of Medical Things (IoMT) moniker. According to [1], an IoMT platform is a smart system that mainly comprises: **i)** sensors and electronic circuits to acquire biomedical signals from a patient, **ii)** processing unit(s) to process the biomedical signals, **iii)** a communication device to transmit the biomedical data over a network, **iv)** a temporary or permanent storage unit, **v)** a visualization platform with artificial intelligence schemes to further elaborate data. For safety monitoring purposes, a considerable number of wearable IoT devices, such as smart bracelets, smartwatches, heart rate monitor, etc., can be leveraged to monitor a range of vital parameters (e.g., heart rate, blood pressure, glucose rate) [2]. Data generated by such devices can be exploited using advanced mechanisms, in order to avoid critical situations or even death, due to worsening of pre-existing conditions. This way, it is possible to provide a valid system to support nursing activities, with powerful

tools that, through continuous monitoring, address priorities to solve critical situations. Focusing on enabling efficient health monitoring within hospital premises, this paper proposes a smart solution to monitor the well-being of patients. The main idea is therefore to design an ICT system able to manage and configure wearables and other useful IoT resources, equipped with sensors probing physiological parameters. In this arrangement, such IoT devices are able to exchange data with a Cloud platform, while equipped with enough resources to run optimized Machine Learning (ML)-based workloads locally, when needed. In particular, our idea is to provide a monitoring system meant to be up and working in next to no time. The aforementioned achievement is possible thanks to limited prerequisites to activate inference, albeit with sub-optimal (initial) accuracy, and able to steadily improve capabilities and efficiency over time using a Distributed Incremental Learning (DIL) approach applied to ML techniques. Indeed, the system supports automatic updates for ML models running on wearable IoT devices thanks to the robust mechanisms for remote code injection provided by Stack4Things, an I/Ocloud-driven Things-as-a-Service platform [3]. The contributions of this paper are: **i)** an innovative approach to enable DIL, and to minimize the effect of the so-called “Catastrophic Forgetting”; **ii)** a fully automated solution managing remote injection of ML models to support dynamically improved and up-to-date inference at the Edge; **iii)** an e-Health case study based on DIL, where efficiency depends on periodically incoming (labeled batches of) datapoints. The proof-of-concept built to satisfy the claim of this work is called *Distributed Incremental Learning on Computing Continuum (DILoCC)*. The paper is structured as follows. Section II introduces some related works. Section III presents the architecture of the system. Section IV describes data lifecycle in the system. Section V discusses the methodology and use case presented. Section VI provides the results of the experiments we conducted to evaluate the efficiency of the system. Section VII closes the paper and provides hints about future works.

II. BACKGROUND AND RELATED WORK

A. Patients' monitoring in the healthcare domain

With the large amount of data collected daily in healthcare, the interest and engagement of Big Data scientists is steadily on the rise. Considerable attention is devoted to the Internet

of Medical Things (IoMT), which plays a key role in patient monitoring in the healthcare industry [4], trying to leverage existing technology [5] to address the challenges of cost, care and minimization of hospital stay [6]. In this context, the interaction between Wearable technology, Cloud computing, and Artificial Intelligence, and their use as decision support systems [7], exploits the ability to predict risk situations that allow medical staff to take action promptly. A number of works have been carried out for the continuous and real-time monitoring of patients in the hospital using the aforementioned technologies, ensuring 24-hour care. Authors in [8] proposed a diagnostic model using a deep neural network (DNN) running on edge computing facilities, using data from IoMT devices. Authors in [9] proposed a “diet recommendation system” for patients, based on IoMT devices and use of machine learning techniques. In [10], the authors created an automatic “real-time heart disease” data analysis model by integrating ensemble deep learning models into IoT-Edge devices to ensure care continuity for their patients. Despite these research efforts to ensure full assistance to their patients, even with excellent performance, they do not fully guarantee to run always and in real-time the best predictive model, considering the high variability of data continuously incoming. In fact, with each arrival of new data, there is the need to re-train the model from scratch to obtain performance improvements, trading off latency, energy consumption and execution time, not even factoring in the costs of collecting new data in clinical trials for patient monitoring. In this context, the data manager seems suitable for the role of supervisor in the clinical trial process [11] that could help in the collection of data over time, so an incremental approach could help ML systems to increase their performance over time.

B. Incremental Learning

Incremental learning (IL) is receiving increased attention in recent years, especially in healthcare applications for which it is becoming necessary to process large and complex data flows by constantly improving the ability of Artificial Intelligence (AI) tools [12]. IL enables a machine learning (ML) or deep learning (DL) model to *continuously* train (e.g., learn from a data stream) over time [13], trying to emulate humans in acquiring knowledge, developing it and transferring ‘experience’ continuously throughout its ‘existence’ [14]. In contrast to a “classical” ML/DL approach, where all data must be available upfront, already in the training phase, IL uses (constantly) incoming data to progressively update the models; this collection of data is used (and can be discarded) to improve knowledge of the pre-existing model without (re)building the new models from scratch, by reducing usage of memory resources [15]. In the Cloud computing era there are limitations to be considered, mostly due to the latencies caused by centralized data processing. Edge computing and AI forge an interesting mix in trying to solve this problem, as data processing is carried out directly on the end devices where possible, providing some degree of ‘intelligence’ at the edge. It is possible to observe three scenarios that may

arise in real-world IL applications [16], where the data are presented in successive/incremental batches, according to the type of scenario: *first*, where after training on a finite number of classes, the new instances belong to one of the classes previously identified, and the new model must continue to recognize the patterns observed previously; *second*, where the model is presented with unforeseen new classes (different from the ones learned so far) and the model must keep the already acquired accuracy while learning the new ones; *third*, in which additional data batches can belong to a new class, or a previously seen one.

Our approach follows the first criterion (we may define it as “instance-based” incremental learning), which is suitable in situations where (like in hospital scenarios) there is the need to learn and transfer knowledge over time, also based on human feedback. In such a context, we demonstrate, in our Edge/Fog computing architecture, following a *naive* [17] baseline strategy, how the model, implemented on an IoT device, improves its performance whenever a batch of data comes in, by limiting the problem of the “catastrophic forgetting”¹ [18], [19]. In addition, through the interaction between IoT devices and the Cloud Continuum system, we show how we can steadily improve, and keep current, the best model fed to the network and, through data batch storage, we can train ML/DL models on large datasets, or even use the entire dataset to process and train different ML/DL algorithms in the future. Many studies show how IL approaches can help those real-time systems where data is dynamic and changes are frequent, by trying to react quickly and adapt the current model to fresh situations without losing acquired ‘expertise’. For example, authors in [20], propose a method of IL for human activity recognition through the use of probabilistic neural networks and an adjustable fuzzy clustering algorithm, while the authors in [21] propose an IL system based on a regression analysis for the on-line prediction of hospitalization duration/inpatient length of stay (LOS)² and the use of resources. This way, they are demonstrating how classic batch learning algorithms are unrealistic in environments where data are dynamic and subject to varying availability over time.

C. Computing Continuum

An IoMT ML/DL-based system may suffer from several issues: latency, error mitigation, device management and coordination, storage (and sharing) of hyperparameters and dataset, etc. Even if latency alone may be tackled by shifting AI-related processing (at least partly) to the Edge, the other issues require a more fine-grained solution that requires the introduction of other elements in the architecture. The complexity and burden due to these aspects is delegated to some powerful system as, for example, a Cloud instance; the approach to combine

¹In Incremental Learning theory, the term Catastrophic Forgetting refers to the phenomenon of performance degradation at inferring previously learned instances, suffered by a Neural Network when trained on new samples.

²Length Of Stay (LOS) refers to the duration of hospitalization for an individual event. The term *inpatient* refers to the patient who resides in the hospital for treatment.

the ability to run lightweight and localized applications at the Edge with the high-capacity processing ensured by the Cloud, including so-called Fog computing infrastructure, is called Computing Continuum (CC), and it is increasingly adopted, as proven by a recent survey [22]. The CC approach is being increasingly adopted in AI-based applications, as described in [23], where the system is supporting the relaying of heavy computation tasks from the Edge to the Cloud. Another interesting work presented in [24] is related to a system exploiting CC facilities to evenly distribute computation according to the output coming from a Markov Decision Process, devoted to identifying the best container deployment site. We focus our attention on the latter two works because in some way they partially address what this work aims to do: on the one hand, the delegation of some task from the Edge to the Cloud and, on the other, a central orchestration system able to distribute the computational load in the CC. On the premise of the previously described limitations, this paper contributes to progress the field by: 1) investigating which mechanisms are needed to actually implement a Computing Continuum (CC) approach for ML workloads; 2) designing and implementing a Distributed Incremental Learning (DIL) workflow; 3) managing remote injection of ML models; 4) demonstrating the validity of the proposed solution with an experimental setup in a clinical setting.

III. ARCHITECTURE

A. System Description and Components

The presented architecture called **DILoCC** is meant to exploit the advantages provided by a Cloud-based system, (e.g., apparently limitless computational and storage capacity), and by resources at the Edge, namely the opportunity to host inference duties and keep these close to the data source (e.g., the sensors) avoiding the aforementioned latency issues. The Cloud acts as an orchestrator but, at the same time, it is tasked with monitoring and presentation duties. As in Fig. 1 one component is executed on the Edge side (Lightning-Rod), and some other components run on the Cloud side (IoTronic, Swift, Neutron and Nova). Lightning-Rod and IoTronic as designed in the Stack4Things (S4T) framework [25], a research project aimed at extending the broadly adopted open source Cloud management system OpenStack, to support the management of IoT deployments akin to datacenter-class infrastructure.

Lightning-Rod (LR) is the S4T agent running at the Edge. The agent can run on any plain, or embedded, Linux-based environment, and is meant to manage IoT devices and single-board computers, including mobiles. It can actuate the orchestration commands received from IoTronic, and the deployment of multiple plugins, concurrently, is also possible. A plugin executes under one of the runtime environments (e.g., Python or NodeJS) available in the platform, and may leverage sensors onboard the device, (pre)processing any data collected through it. In DILoCC, one of the injected plugins is used to run the model: this plugin collects data from sensors (to prepare the training batches), and it uses the batch thus created to train the model, and then the (plugin-hosted) model as well processes it

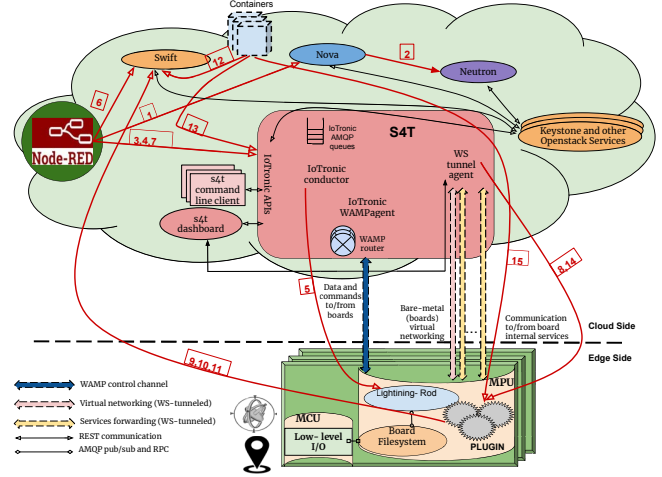


Fig. 1. DILoCC general architecture.

as (inference) input. Another injected plugin is used to support the functionality described in the workflow III-B2.

IoTronic instead represents the Cloud side of the framework, that exploits communication patterns based on WAMP (Web Application Message Protocol³) to control devices. IoTronic is adherent to the standard architecture of OpenStack services, and for this reason it is fully compatible with other subsystems than can all easily interoperate across an OpenStack Cloud instance. In the proposed architecture this component is tasked with the lifecycle management of Edge devices and hosted resources, and it receives (device lifecycle) instructions from the virtualized server, hosted by another subsystem in the architecture, Nova. The virtualized server is the element in charge of translating administrative commands into device lifecycle instructions. Other Cloud-side components belong to the Openstack universe of projects, the only exception being NodeRED, a management dashboard, for, e.g., adding new devices, forcing workflow activations, monitoring, and so on. The following subsystems, in particular, provide facilities to the DILoCC system: **Swift** [26] aiming to provide object storage facilities, thus acting as a repository for plugins, models and datasets; it interacts with the server hosted by Nova, providing upon request the object to be pushed to the Edge device. **Nova** [26] is meant to provide computing on-demand facilities to its users. In the proposed architecture, this component hosts the server process described in workflows III-B1 and III-B3; at the same time, it exposes the services invoked by the workflow III-B2. **Neutron** [26] is devoted to the management of virtual networks. In DILoCC, it serves the requests coming from IoTronic for network management atop IoT boards and, during setup of the DILoCC system itself, Neutron interacts with Nova to provide network connectivity to the VMs and Containers.

³WAMP is defined as: “WAMP is an open standard WebSocket subprotocol that provides two application messaging patterns in one unified protocol: ‘routed Remote Procedure Calls’ and ‘Publish & Subscribe’”.

B. Workflow

We have identified three main workflows, which are the most representative to exploit the Cloud Continuum facilities and make them available for the application, through the interaction occurring between the Cloud and the IoT devices (represented by red arrows in Fig. 1). This way, the system functionalities support IL by providing coordination and fail-safe (model) recovery.

1) *Device enrolment*: The workflow used to introduce a new device in the system is relevant because it shows how the complexity of a similar action is managed with a few simple actions. The first step consists in the preparation of the IoT device; this is managed by a “device-centric sensing” [27] system running on the Cloud, specifically based on Stack4Things, an *unofficial* project within the Openstack ecosystem, acting as a reference implementation for the I/OCLOUD paradigm [3]; the first step consists of installation and configuration of Lightning-Rod component to connect with the system. This corresponds to action 3, that represents an interaction occurring between *NodeRED* and *S4T*. The *registration*, *activation*, and *exposition* of IoT-hosted services all belong to this step. When the device is connected, the administrator, according to the physical device configuration, can inject on the device a compatible IL model, as represented by actions 4, 5, and 6. A detailed flowchart for this workflow is depicted in Fig. 2.

As said when presenting the architecture, the model will receive its input from the devices managed by the IoT (Sensor and Actuator), this means that a modification of the model is needed. The last step, before model and dataset are injected (actions 7, 8 in Fig. 1) on the device, is related to the verification of an enhanced (e.g., higher accuracy) version of the model; this is possible because for each model the enhanced version coming from the incremental learning activity made by the devices is stored. The connection among the template model and their incremental trained version is possible through the association of a UUID that correlates with the accuracy reached in the test phase done by the devices (see workflow III-B2) and stored on the Cloud by one of the two workflows shown below.

2) *Autonomous (IoT-driven) model sanity checks*: The plugin injected on the IoT devices is tuned to execute an accuracy verification every hour. Each phase of the training may be followed by a test one, in which the model verifies the result of the IL step through the computation of the model’s accuracy and comparing it with the one previously computed. At this point, if accuracy has changed, the plugin compares the model accuracy obtained from model verification (action 9, as depicted in Fig. 1) with the enhanced version of the model stored on the Cloud-hosted repository. If the accuracy value retrieved, subtracted by a fixed amount (set in the plugin when injected) is higher than the current model accuracy, the system begins the recovery phase.

The recovery phase is an action that forces the Cloud to improve the model running in the device by automatically injecting a better model and the corresponding training batch (that will be deleted after training), see action 11. Conversely,

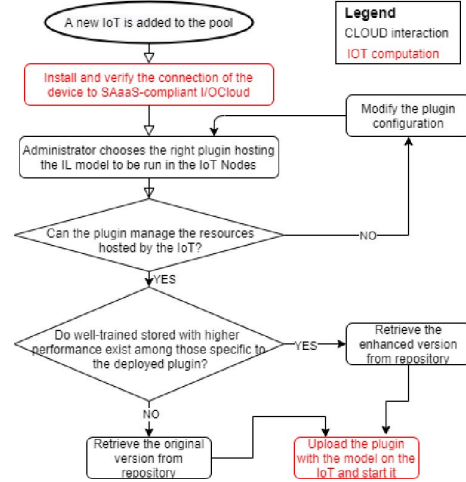


Fig. 2. Enrolment of new IoT node to the pool.

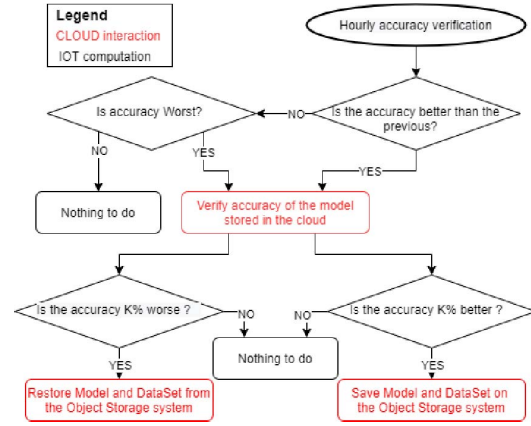


Fig. 3. Automatic model verification done by the IoT.

if the value computed for the model running on the IoT is higher than the one retrieved from the Cloud plus the fixed amount, then the plugin forces the Cloud to save in its repository a snapshot of the actual version of the plugin’s model and its dataset, see action 10. A detailed flowchart for this workflow is depicted in Fig. 3.

3) *Periodic (Cloud-driven) model verification*: The last workflow analyzed regards the Cloud-based verification of the system’s accuracy. The Cloud checks periodically if the integrity and functionality of the model are preserved, and at the same time, it verifies if the version of the model running on the devices is the best one available in the system. This mechanism is invoked by the Cloud periodically at intervals defined by a configuration parameter set by the administrator (by default, on a daily basis). Even if the interval definition is coming from system configuration, the Cloud hosts a monitoring process able to modify this value according to the frequency of model “restore” requests received from the IoT devices, by:

- reducing the interval, if all the IoTs invoke at least one time the model “restore” function (one of three closing

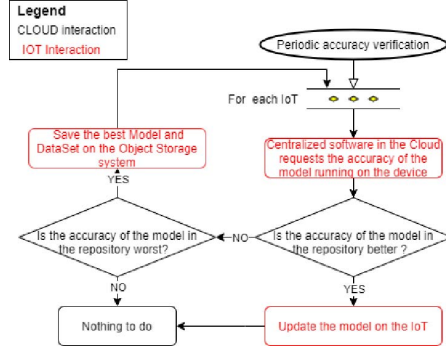


Fig. 4. Automatic model verification done by the Cloud.

conditions shown by Fig. 3) between two occurrences of the Cloud-driven model verification;

- extending the interval when, in a complete periodical check, each of the IoT devices' verification reaches the "Nothing to do" state, without model updates on an IoT (see Fig. 4).

Cloud-driven model verification solves situations in which a model reaches a condition where its performance cannot be improved with the data received as input. Inside a polling loop, the Cloud requests the value of accuracy reached by each plugin hosted in the IoT devices, as depicted in Fig. 1 with actions 13, 14. This way, the plugin running the model can compare this value with the one stored on the repository, as shown with action 12 in Fig. 1. Similarly to the last part of the workflow described in Section III-B2, also in this case the accuracy is tested to identify if a restore action is needed for the model running in the IoT (action 11), or to upload the improved model just refined to the Cloud repository (action 10). The accuracy verification workflow finishes when the workflow reaches the "Nothing to do" state for all the devices. The workflow just described is shown in Fig. 4.

IV. DATA LIFECYCLE

A. Management of Model and Dataset

Dataset and model, as discussed above, are stored in the system repository (Swift) and there represented as *blobs* of data, each object corresponding to a single blob. Objects regarding the same AI-application are stored in a Swift-Container and accessed through requests addressing the name of the object requested. The system running on the VM (or Container) hosted by Nova interacts with the repository downloading and uploading these files according to workflows III-B2 and III-B3. For this purpose, it has to maintain the accuracy value to decide about workflows. It appears that this system is able to simultaneously manage multiple categories of IoT devices and IL-based applications working in the same environment; this imposes the storage of another information, the UUID of the application, that, together with the accuracy, represents the coordinates needed to univocally identify the right Container to address and the right Objects to be retrieved.

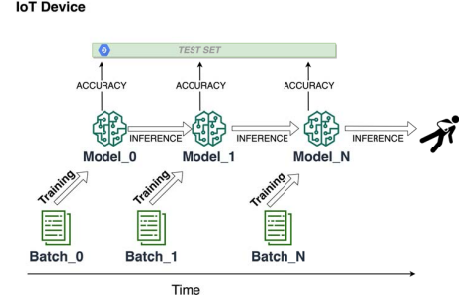


Fig. 5. Incremental learning scheme in a single IoT device.

B. Feeding Inputs to the Model

In the Edge devices, the model is able to obtain the input from the sensors and actuators attached to the devices thanks to the plugin that exposes the data coming from *pins* in the form of a file of the filesystem. Indeed the agent can interact with the (board) pin through a pseudo-file exposing the value being measured (sampled) at the time of (file) access. The plugin, through the agent APIs, interacts with the board-hosted resources according to the aforementioned filesystem-based abstractions. This way data available to the pins will be elaborated to produce an output to be delivered to the reporting system (e.g., a NodeRed-based GUI exposed by the same server coordinating the devices) through RESTful interactions.

V. METHODOLOGY AND USE CASE

In this section, we provide more details about the methodology and the dataset used by showing how the data flows in the system. IL algorithms, thanks to their real-time continuous processing capability, are suitable for learning by allowing models to adapt directly to data flows. Our focus is on large-scale processing and on providing IL with incremental batches of data from the IoT devices. Our method, in contrast to the classical ML/DL methods, allows to deal with the context of learning from real-time data flows, with the assumption that the data is available in a certain form and quantity, thanks to the *data manager* role, i.e., personnel which becomes especially useful in the data collection and the management of clinical trials. This allows us to use the data as soon as it is available, leading to more up-to-date and accurate models, whilst also reducing data storage costs. We demonstrate that our approach is scalable and effective by solving a range of supervised dataset-based classification tasks.

We define an IL algorithm as one that generates a given stream of training data $Ds_1, Ds_2, Ds_3, \dots, Ds_n$ and a sequence of models $M_1, M_2, M_3, \dots, M_n$, where the last model depends on model M_{i-1} and the size of the data stream is limited to enable perpetual training on *Internet of Medical Things* (IoMT) devices with limited resources. As use case, we refer to the scenario discussed in [28], where the IoMT are wearable devices connected to Raspberry Pi (model 3), meant to monitor the activities of patients in a hospital environment. Our IL approach is exemplified in Fig. 5, where we have a potentially infinite sequence of tuples $Ds_i = (x_i, y_i)$, which

one after the other reach the IoT device. The learning goal is to provide the corresponding label for a given data stream that is unknown to the M_i model. According to the scheme, all the intermediate models are considered for the evaluation of subsequent performance, but each of them is equipped only with their examples and not with the entire dataset. Prediction is performed based on the previous knowledge of the model, and accuracy is computed on a separate test dataset. The final accuracy of an incremental algorithm is related to the average generalization capability of all models built.

We used a *naive* strategy that refines the model as new batches arrive, through simple regularization rules, to avoid overfitting and improve generalization. This method usually suffers in the presence of heterogeneous batch sizes. Therefore, for this setting, we decide to distribute same-size data batches to the IoMT devices to avoid this kind of problem. From a practical IoMT point of view, it is assumed that data batches flowing inside the IoT devices are labelled, independent, equal in size and not necessarily equally distributed labels-wise, and that all the IoT devices are equipped with an initial multi-layer neural network model chosen according to the prediction of a specific multi-classification task.

As an initial model, we use a deep neural network (DNN) with 4 hidden layers between the input and output. This type of architecture is sufficient to represent the relationship between input and output and is compliant with all those situations in which very complex non-linear relationships are present. The input layer is composed of the measurements collected by the sensors, while the output consists of the different activities (labels) we want to classify. The same hyperparameters have been used for each incoming batch of data. In particular, the parameters used are:

Hidden layers: $H1$ (25 neurons), $H2$ (20 neurons), $H3$ (15 neurons), $H4$ (10 neurons);

Learning rate (lr): 0.01 for all data batches;

Activation function: *Relu* for the initial layer and *Softmax* for the output layer;

Epochs: 20 epochs have been used for training in all batches;

Weight decay: the regularization constant $L2$ was set at 0.01 (default value);

Optimizer: *Adam* optimizer, with learning rate set to 0.01.

Data batches: the activities stored per class are equal to 300 in the first batch of data (1800 samples, 6 different activities/labels to be categorized), while for the following batches we used an average of about 100 example per class (it varies from batch to batch) for a total fixed size of 500 rows.

For our purpose, we used the original dataset that was uploaded to the public repository in [29]. This way, it is possible to focus on the IL advantages, compared against a fixed model, to deal with possible alteration of the working conditions (i.e., sensor's performance degradation, misplacing of a beacon, and so on). To validate the proposed approach, we have randomly split the dataset into 10 batches of data of equal size with 500 data points for the training phase of each model and 600 data points for the test set. We used a

classic sampling method that returns a random sample of rows through the *Pandas* library.

VI. EXPERIMENTAL RESULTS

We have evaluated our approach through various experiments conducted from different perspectives. First, we test the functioning of IL on a single IoT device with 15 experiments, then, we evaluate the system by testing on 10 different IoT devices interconnected to the Cloud system, and evaluating the performance in terms of accuracy, inference time, and speed of convergence (toward steady state). In particular, we test the functioning of our incremental approach on data that have been used for health monitoring of hospitalized patients. We specifically consider the problem of the onset of bedsores using wearable technology and DL techniques in a supervised approach [28]. Our method also ensures that when new input data arrives, the model extends its knowledge by allowing the system not to perform the training phase from scratch with the entire dataset, but to consume only data that becomes gradually available over time.

As shown in Table I and Fig. 6, we notice how this approach succeeds in providing satisfactory results in our 15 experiments carried out on a single device. We see how knowledge improves, the more data batches come in. We note that, albeit a little discontinuity can be observed in the growth of the model, showing signs of forgetting and degradation of the accuracy, on average it monotonically increases, demonstrating that the model always works, in the long run.

Regarding the metrics used, we report the accuracy and the relative loss, that are always computed on the data constituting the test set, which is out of the model training process. From an architectural point of view, to analyze how our system works in the real-world scenario, we have deployed

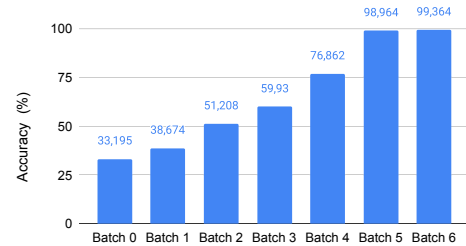


Fig. 6. Average accuracy after incremental data batches.

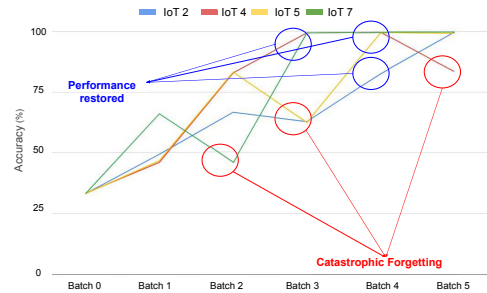


Fig. 7. DILoCC's effect on accuracies of IoT nodes affected by *Catastrophic Forgetting* in Table II.

TABLE I
EXPERIMENTS CARRIED OUT ON A SINGLE DEVICE.

Run Id	Metrics	Batch0	Batch1	Batch2	Batch3	Batch4	Batch5	Batch6
1	Accuracy(%)	33.33	47.66	50.00	57.66	65.83	99.00	99.50
	Loss	4.118	3.488	1.814	1.521	1.454	0.345	0.205
2	Accuracy(%)	33.00	33.33	49.50	61.83	82.33	99.33	99.50
	Loss	5.774	5.376	3.041	2.469	1.183	0.249	0.220
3	Accuracy(%)	33.16	33.33	49.16	65.16	82.50	98.66	99.50
	Loss	5.022	4.269	2.469	2.441	1.386	0.264	0.235
4	Accuracy(%)	33.33	50.00	50.50	60.83	82.33	99.16	99.66
	Loss	3.765	3.715	1.980	1.557	1.232	0.225	0.218
5	Accuracy(%)	33.16	33.16	49.33	64.33	81.83	99.16	99.33
	Loss	6.902	5.952	2.532	2.087	1.565	0.232	0.225
6	Accuracy(%)	33.33	50.00	61.83	62.00	82.50	99.33	99.66
	Loss	3.952	3.692	2.588	2.332	1.780	0.254	0.233
7	Accuracy(%)	33.33	50.00	63.00	66.00	82.16	99.00	99.16
	Loss	4.609	4.002	2.136	1.341	0.907	0.208	0.202
8	Accuracy(%)	33.16	33.33	49.33	64.50	82.33	99.00	99.66
	Loss	5.219	4.334	2.346	2.171	1.979	0.257	0.235
9	Accuracy(%)	33.16	33.16	49.16	62.33	82.50	99.00	99.16
	Loss	3.941	2.684	1.750	1.460	1.217	0.213	0.211
10	Accuracy(%)	33.00	33.16	49.16	65.33	65.49	98.83	99.16
	Loss	4.322	4.763	2.605	2.482	1.884	0.249	0.233
11	Accuracy(%)	33.16	33.33	32.83	48.33	65.99	98.83	99.00
	Loss	4.322	3.571	2.332	2.326	2.321	0.264	0.261
12	Accuracy(%)	33.16	33.16	49.33	44.33	65.99	98.50	99.16
	Loss	5.758	5.068	3.354	2.117	1.794	0.288	0.271
13	Accuracy(%)	33.16	33.16	49.33	46.83	65.99	98.16	98.85
	Loss	5.552	4.763	2.50	2.00	1.385	0.259	0.239
14	Accuracy(%)	33.16	33.33	49.50	64.00	82.66	99.00	99.50
	Loss	4.770	3.863	2.105	1.585	0.861	0.237	0.226
15	Accuracy(%)	33.33	50.00	66.16	65.50	82.50	99.50	99.66
	Loss	3.379	2.780	1.673	1.168	1.011	0.245	0.234
Avg (%)		33.195	38.674	51.208	59.930	76.862	98.964	99.364
Std.Dev.(+/-)		0.1124	7.9672	7.7813	7.3209	8.0571	0.3411	0.2658

TABLE II
EXPERIMENTS CARRIED OUT ON 10 IoT DEVICES BY USING SHUFFLED DATA BATCHES.

IoT Device ID	Metrics	Batch0	Batch1	Batch2	Batch3	Batch4	Batch5
1	Acc.(%)	33.16	49.33	82.66	82.83	99.16	99.50
	Loss	4.037	2.712	0.647	0.929	0.306	0.221
2	Acc.(%)	33.16	49.33	66.66	62.83	82.66	99.66
	Loss	5.196	2.777	2.712	1.528	0.892	0.213
3	Acc.(%)	33.00	33.33	46.16	82.66	99.16	99.33
	Loss	4.830	4.371	1.306	1.073	0.305	0.218
4	Acc.(%)	33.33	45.99	82.83	99.30	99.66	83.33
	Loss	5.045	3.009	0.901	0.279	0.230	0.756
5	Acc.(%)	33.16	46.66	83.16	62.50	99.50	99.16
	Loss	3.578	2.912	0.493	2.712	0.273	0.228
6	Acc.(%)	33.00	65.99	83.00	99.16	99.16	99.50
	Loss	4.276	2.182	0.517	0.265	0.259	0.243
7	Acc.(%)	33.16	65.99	45.99	99.33	99.66	99.66
	Loss	5.128	2.214	1.404	0.328	0.229	0.214
8	Acc.(%)	33.00	33.33	82.66	82.66	99.50	99.50
	Loss	4.895	3.718	0.645	1.219	0.272	0.220
9	Acc.(%)	33.00	49.33	66.66	99.33	99.66	99.66
	Loss	3.855	2.753	1.769	0.275	0.231	0.214
10	Acc.(%)	33.33	47.16	82.83	82.66	99.16	99.16
	Loss	4.466	3.351	0.950	1.076	0.239	0.316

10 different IoT devices. Starting from the first model built on the data of the *batch0*, we select the best model (in this case belonging to the IoT device 10 from Table II) with better accuracy and less loss, as usual tested on the test data. Then, we share/distribute it on all the other IoT devices. Afterwards, we re-train all the devices with the data of (a shuffled instance of) the next batch in line (in this case, *batch1*), and test their performance. We have repeated this procedure for all the devices with subsequent data batches. Thus, for each batch, it is possible to identify the model (i.e., the corresponding IoT device generating it) with the best accuracy (the ones highlighted in bold in Table II) at each step (i.e., processed batch). From the Table we can notice that, albeit some IoT devices in *batch2* worsen their performance, thus proving to be inefficient in terms of knowledge growth, there are models within the system that improve the system performance when new batches of data arrive. In this case, we proceed with the *batch3*, abandoning the best model seen in *batch2*.

We demonstrate how the facilities provided by *DILoCC* can support the *DIL* approach performed on the devices. This way, even if at a certain step there is a slight deterioration in the performance of some models (e.g., *catastrophic forgetting*),

thanks to the performance verification of the model (see workflows III-B2 and III-B3), the best model is always distributed, and the steady-state is reached in a shorter amount of time. Fig. 7 shows the accuracy behavior of the models affected by the *catastrophic forgetting* phenomenon. It can be observed here how the *DILoCC*-based approach restores the accuracy of the models when a degradation occurs. Indeed, as shown in Fig. 7, the models suffer from *Catastrophic Forgetting* in different moments: i) IoT7 after *batch1*, ii) IoT5 and IoT2 after *batch2*, and iii) IoT4 after *batch4*. These degradations in accuracy are recognized during the next batch verification phase (e.g., IoT7 is recognized in *batch2*). The *DILoCC*'s countermeasures refer to the workflows III-B2 and III-B3 that can promptly restore the model performance, as it can be observed from the accuracy value measured in the next batch (e.g., the model's accuracy of IoT7 is restored during the *batch3* verification phase). The system reaches convergence as soon as *batch4* arrives, with an accuracy of 99.66%. The format of the trained model has a size of 49 KB, which is quite large, considering that it has to be imported and exported from Raspberry devices. The average inference time is 0.003 seconds per sample, which ensures usability in a realtime scenario. Considering that the batches of data from the IoT devices are not uniformly distributed (the labels' distribution changes too), by evaluating this type of scenario we understood that the steady state can be reached only if the requirements and assumptions on the size of the batches is duly satisfied. We conclude that the convergence speed of the model training is strongly correlated to a) the number of devices belonging to the system and b) the quality of the data batches.

VII. CONCLUSION AND FUTURE WORKS

In this paper, we presented the design and implementation of *DILoCC*, a methodology and framework for (Distributed) Incremental Learning across the Computing Continuum. The system mitigates the issues deriving from Catastrophic For-

getting by leveraging automatic update distribution of ML models running on IoT devices, enabled by robust remote code injection facilities provided by the Stack4Things platform. Performance parameters were analyzed, in a e-Health context, to demonstrate how combining DIL with CC infrastructure may deal successfully also with realtime monitoring, allowing to develop adaptable self-tuning solutions to operational changes.

Regarding future developments, we are also going to target the Federated Learning (FL) approach [30] beyond DIL, in order to widen the scope of application for the system. Furthermore, we want to further investigate the Catastrophic Forgetting phenomenon and decrease its impact by applying more suitable model regularization.

Another prospective development is related to health monitoring without any involvement of medical personnel, thus moving towards a *fully* unsupervised system, self-adapting to variable working conditions. We are also planning to extend the management facilities provided through the DILoCC infrastructure to identify, manage, and automatically correct, with special policies, the situation in which sensor drifts and defects alter the device's behavior. The aforementioned advancements may be achieved by refining the Fog computing layer in our architecture, to contribute with further computation and control facilities to the infrastructure, as a remediation against any limited availability, and/or sensor response degradation over time.

REFERENCES

- [1] S. Ramson and D. J. Moni, "A case study on different wireless networking technologies for remote health care," *Intelligent Decision Technologies*, vol. 10, no. 4, pp. 353–364, 2016.
- [2] M. Chen, Y. Ma, Y. Li, D. Wu, Y. Zhang, and C.-H. Youn, "Wearable 2.0: Enabling human-cloud integration in next generation healthcare systems," *IEEE Communications Magazine*, vol. 55, no. 1, pp. 54–61, 2017.
- [3] D. Bruneo, S. Distefano, F. Longo, G. Merlino, and A. Puliafito, "IOcloud: Adding an IoT dimension to Cloud infrastructures," *Computer*, vol. 51, no. 1, pp. 57–65, January 2018.
- [4] G. J. Joyia, R. M. Liaqat, A. Farooq, and S. Rehman, "Internet of medical things (iomt): applications, benefits and future challenges in healthcare domain," *J Commun*, vol. 12, no. 4, pp. 240–247, 2017.
- [5] W. Raghupathi and V. Raghupathi, "Big data analytics in healthcare: promise and potential," *Health information science and systems*, vol. 2, no. 1, p. 3, 2014.
- [6] Sungmee Park and S. Jayaraman, "Enhancing the quality of life through wearable technology," *IEEE Engineering in Medicine and Biology Magazine*, vol. 22, no. 3, pp. 41–48, 2003.
- [7] S. Hiremath, G. Yang, and K. Mankodiya, "Wearable internet of things: Concept, architectural components and promises for person-centered healthcare," in *2014 4th International Conference on Wireless Mobile Communication and Healthcare (MOBIHEALTH)*, 2014, pp. 304–307.
- [8] I. V. Pustokhina, D. A. Pustokhin, D. Gupta, A. Khanna, K. Shankar, and G. N. Nguyen, "An effective training scheme for deep neural network in edge computing enabled internet of medical things (iomt) systems," *IEEE Access*, vol. 8, pp. 107 112–107 123, 2020.
- [9] C. Iwendi, S. Khan, J. H. Anajemba, A. K. Bashir, and F. Noor, "Realizing an efficient iomt-assisted patient diet recommendation system through machine learning model," *IEEE Access*, vol. 8, pp. 28 462–28 474, 2020.
- [10] S. Tuli, N. Basumatary, S. S. Gill, M. Kahani, R. C. Arya, G. S. Wander, and R. Buyya, "Healthfog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated iot and fog computing environments," *Future Generation Computer Systems*, vol. 104, pp. 187 – 200, 2020. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0167739X19313391>
- [11] F. Rico-Villademoros, T. Hernando, J.-L. Sanz, A. López-Alonso, O. Salamanca, C. Camps, and R. Rosell, "The role of the clinical research coordinator–data manager–in oncology clinical trials," *BMC Medical Research Methodology*, vol. 4, no. 1, p. 6, 2004.
- [12] K. Y. Ngiam and I. W. Khor, "Big data and machine learning algorithms for health-care delivery," *The Lancet Oncology*, vol. 20, no. 5, pp. e262 – e273, 2019. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S1470204519301494>
- [13] H. He, S. Chen, K. Li, and X. Xu, "Incremental learning from stream data," *IEEE Transactions on Neural Networks*, vol. 22, no. 12, pp. 1901–1914, 2011.
- [14] P. Joshi and P. Kulkarni, "Incremental learning: Areas and methods-a survey," *International Journal of Data Mining & Knowledge Management Process*, vol. 2, no. 5, p. 43, 2012.
- [15] B. Hammer *et al.*, "Incremental learning algorithms and applications," in *ESANN*, 2016.
- [16] Y. Luo, L. Yin, W. Bai, and K. Mao, "An appraisal of incremental learning methods," *Entropy*, vol. 22, no. 11, p. 1190, 2020.
- [17] S. Ren, Y. Lian, and X. Zou, "Incremental naïve bayesian learning algorithm based on classification contribution degree," *JCP*, vol. 9, no. 8, pp. 1967–1974, 2014.
- [18] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017. [Online]. Available: <https://www.pnas.org/content/114/13/3521>
- [19] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," ser. *Psychology of Learning and Motivation*, G. H. Bower, Ed. Academic Press, 1989, vol. 24, pp. 109 – 165. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0079742108605368>
- [20] Z. Wang, M. Jiang, Y. Hu, and H. Li, "An incremental learning method based on probabilistic neural networks and adjustable fuzzy clustering for human activity recognition by using wearable sensors," *IEEE Transactions on Information Technology in Biomedicine*, vol. 16, no. 4, pp. 691–699, 2012.
- [21] S.-K. Ng, G. J. McLachlan, and A. H. Lee, "An incremental em-based learning approach for on-line prediction of hospital resource utilization," *Artificial Intelligence in Medicine*, vol. 36, no. 3, pp. 257–267, 2006.
- [22] L. Bittencourt, R. Immich, R. Sakellariou, N. Fonseca, E. Madeira, M. Curado, L. Villas, L. DaSilva, C. Lee, and O. Rana, "The internet of things, fog and cloud continuum: Integration and challenges," *Internet of Things*, vol. 3–4, pp. 134 – 155, 2018. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S2542660518300635>
- [23] D. Aguiari, A. Ferlini, J. Cao, S. Guo, and G. Pau, "Poster abstract: C-continuum: Edge-to-cloud computing for distributed ai," in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2019, pp. 1053–1054.
- [24] P. Kochovski, R. Sakellariou, M. Bajec, P. Drobintsev, and V. Stankovski, "An architecture and stochastic method for database container placement in the edge-fog-cloud continuum," in *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2019, pp. 396–405.
- [25] F. Longo, D. Bruneo, S. Distefano, G. Merlino, and A. Puliafito, "Stack4things: An openstack-based framework for iot," in *2015 3rd International Conference on Future Internet of Things and Cloud*, Aug 2015, pp. 204–211.
- [26] "OpenStack documentation [URL]," <http://docs.openstack.org>.
- [27] S. Distefano, G. Merlino, and A. Puliafito, "Device-centric sensing: An alternative to data-centric approaches," *IEEE Systems Journal*, vol. 11, no. 1, pp. 231–241, 2017.
- [28] G. Cicceri, F. De Vita, D. Bruneo, G. Merlino, and A. Puliafito, "A deep learning approach for pressure ulcer prevention using wearable computing," *Human-centric Computing and Information Sciences*, vol. 10, no. 1, p. 5, 2020.
- [29] G. Cicceri, "Human activity registration," Jul. 2019, dataset. [Online]. Available: <https://doi.org/10.5281/zenodo.3351545>
- [30] J. Yoon, W. Jeong, G. Lee, E. Yang, and S. J. Hwang, "Federated continual learning with adaptive parameter communication," *arXiv preprint arXiv:2003.03196*, 2020.